UVM Consolidation

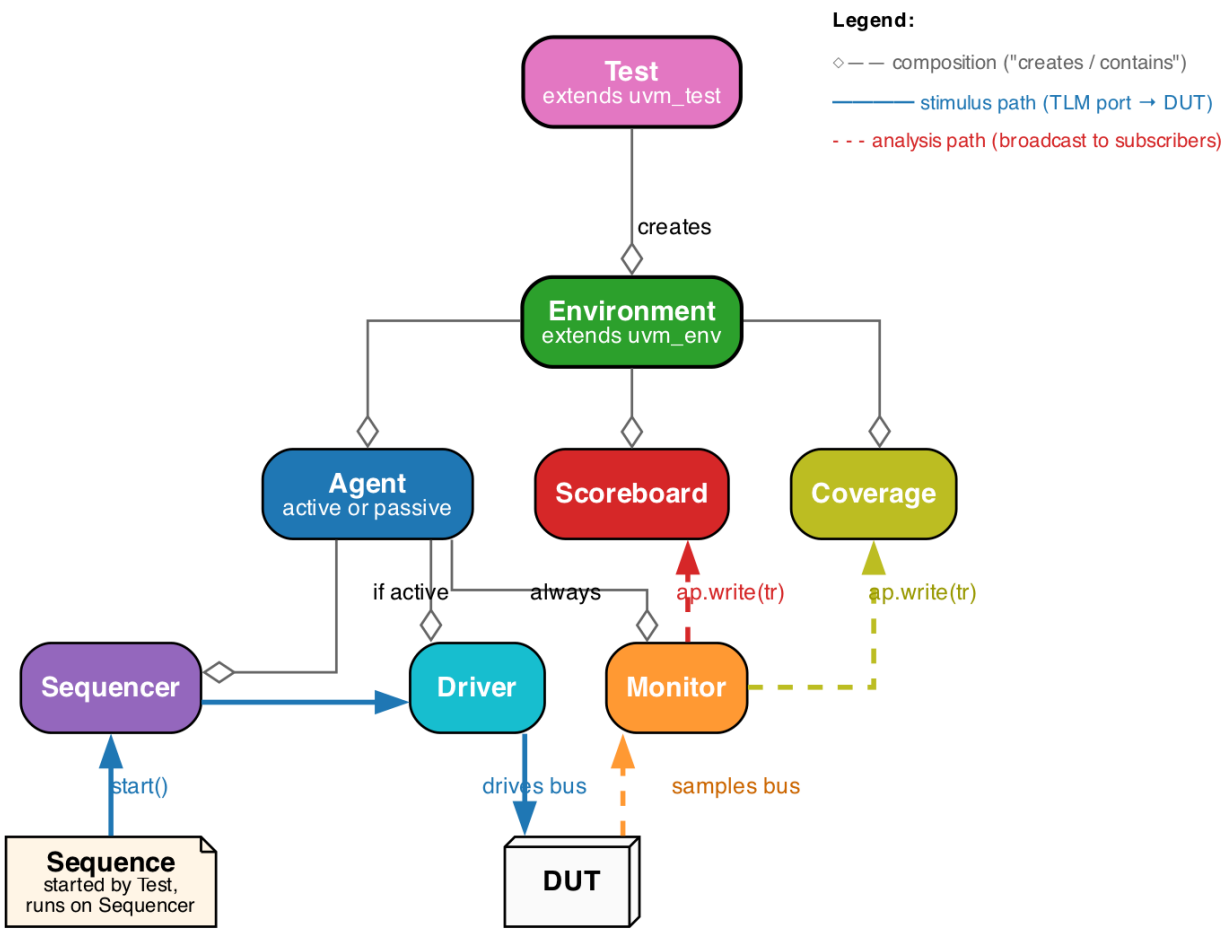
Salemi *UVM Primer* — Chapters 9–15
One structure, one naming convention, one source of truth

Why this document exists. If you've just read Salemi 9–15 for the first time and feel scattered, this is normal. Salemi uses pedagogical names (*tester*, *base_tester*, *random_tester*) that don't match what real teams use (*driver*, *sequence*, *sequencer*). This doc gives you ONE canonical structure with the industry-standard names, plus a translation table back to Salemi.

How to use it. Read it once front-to-back (≈ 30 min). Then keep it open as a reference while you work through the chapter exercises and the week-4 homework. Don't try to memorize — use it to look up.

1. The Canonical UVM Architecture

Every production UVM testbench has the same shape. Memorize this picture — everything else is variations on it.



Two halves to remember:
1. Stimulus path (left side): a *sequence* describes WHAT to send. The *sequencer* arbitrates between sequences. The *driver* talks to the DUT through the BFM/interface.
2. Analysis path (right side): the *monitor* passively observes the bus and broadcasts *transactions* via an **analysis_port** to multiple subscribers (scoreboard, coverage).

2. The 8 Class Types You Need to Know

Memorize this table. Every UVM testbench you'll ever read uses these 8 class types.

Class	Extends	Lives in	Job
uvm_sequence_item	uvm_object	Created on demand by sequence	One transaction (request/response data)
uvm_sequence	uvm_object	Started by test, runs on sequencer	Generates a stream of sequence_items
uvm_sequencer	uvm_component	Inside agent	Arbitrates sequences, hands items to driver
uvm_driver	uvm_component	Inside agent	Pulls items from sequencer, drives DUT
uvm_monitor	uvm_component	Inside agent	Passively samples bus, broadcasts via analysis_port
uvm_agent	uvm_component	Inside env	Bundles drv+sqr+mon (active) or mon only (passive)
uvm_scoreboard	uvm_component	Inside env	Subscribes to monitor, predicts + checks DUT
uvm_env	uvm_component	Inside test	Builds agents + scoreboard + coverage; wires them
uvm_test	uvm_component	Top of UVM tree	Builds env; sets factory overrides; starts sequences

Two families: uvm_object descendants are *data* (short-lived, no hierarchy, no phases). **uvm_component** descendants are *persistent infrastructure* (live the whole sim, parent/child tree, have phases like build_phase / run_phase).

3. Naming Translation: Salemi → Industry Standard

Salemi uses simplified pedagogical names. Real teams use the names below. When you see a Salemi term, mentally substitute the industry-standard one.

What Salemi calls it	Industry-standard name	Notes
tester (ch.10)	≈ driver (no UVM yet)	ch.10's plain class — pre-UVM ancestor of the driver
base_tester (ch.13)	abstract base for driver	Pattern used to enable factory override of stimulus
random_tester / add_tester	different drivers (sometimes different sequences)	In modern UVM the variation lives in *sequences*, not the driver
BFM (interface tasks)	interface + clocking block + driver	Salemi puts *send_op* in the interface; modern UVM puts it in the driver
coverage class (extends uvm_component)	uvm_subscriber #(transaction)	Subscriber is the proper base for analysis recipients
scoreboard (with bfm handle)	uvm_scoreboard with uvm_analysis_imp	Scoreboard subscribes to monitor.ap, NOT direct bfm access
env (in ch.13)	uvm_env	Same name — Salemi follows the standard here
random_test / add_test (ch.11+)	uvm_test subclasses	Same pattern — names match
BFM tasks like send_op()	driver's run_phase logic	In UVM the protocol drives live in the driver, not the interface
@(posedge bfm.done) inside scoreboard	monitor samples + writes to ap	Modern UVM: scoreboard NEVER touches the bus directly

The biggest mental shift: Salemi's *tester* generates AND drives stimulus in one class. Modern UVM splits that into **sequence** (what to send) + **sequencer** (arbitrates) + **driver** (talks to DUT). This split is what enables reusable verification IP.

4. The UVM Phases (in order)

Every uvm_component goes through these 9 phases. UVM calls each phase function automatically — you just override the ones you need.

#	Phase	Type	Direction	What you do here
1	build_phase	function	top-down	Create child components with *type_id::create*
2	connect_phase	function	bottom-up	Wire TLM ports: *ap.connect(imp)*
3	end_of_elaboration_phase	function	bottom-up	Last sanity check before sim starts
4	start_of_simulation_phase	function	bottom-up	Print topology, banner messages
5	run_phase	task	parallel	The TIME-CONSUMING phase. raise_objection here.
6	extract_phase	function	bottom-up	Pull final data out of components
7	check_phase	function	bottom-up	Final assertions / checks
8	report_phase	function	bottom-up	Print pass/fail summary
9	final_phase	function	bottom-up	Last cleanup before $finish

Rules of thumb: always call *super.<phase>(phase)* first. Only run_phase consumes time. Always raise/drop objections in run_phase. The order build → connect → run is the order you'll use 95% of the time.

5. Salemi Chapter Summaries (9–15)

Chapter 9 — The Factory Pattern

- Problem: you can't *new()* classes if you want to swap implementations later.
- Solution: a global **factory** that knows every class type by name and proxy.
- Register every class with **`uvm_object_utils(name)** or **`uvm_component_utils(name)**.
- Build with *name::type_id::create("inst", parent)* instead of new().
- Override at runtime with *set_type_override_by_type* / *by_name*.

Chapter 10 — Object-Oriented Testbench (the bridge to UVM)

- A pre-UVM testbench architecture: a *testbench* class that composes *tester*, *scoreboard*, *coverage*.
- Each holds a *virtual* interface handle to reach the BFM.
- Manual *fork/join_none* orchestrates concurrent execution.
- UVM later replaces this with phases and uvm_env, but the SHAPE stays the same.

Chapter 11 — UVM Tests

- Introduces *uvm_test* as the top-level class — extends uvm_component.
- *top.sv* calls **run_test()**; UVM reads the **+UVM_TESTNAME** plusarg.
- Test's build_phase creates the env. Test's run_phase raises/drops objections.
- Multiple test classes can coexist — pick one at runtime via plusarg, no recompile.

Chapter 12 — UVM Components

- Every persistent TB thing is a *uvm_component* with a parent.
- 9 phases run automatically — you override the ones you need.
- Reporting macros: *`uvm_info* / *`uvm_warning* / *`uvm_error* / *`uvm_fatal*.
- Introspection: *get_full_name()* / *get_parent()* / *get_name()*.

Chapter 13 — UVM Environments

- *uvm_env* composes children and wires their TLM connections.
- Headline pattern: abstract *base_tester* + factory override. Test installs override BEFORE creating env; env asks the factory for base, factory swaps in the concrete subclass.
- Same env, multiple stimulus styles, picked at runtime via the test's *set_type_override*.

Chapter 14 — A New Paradigm

- Wrap-up: the value proposition of UVM is reusable verification IP, factory-driven flexibility, and standardized phasing.
- Sets the stage for sequences (ch.15+) which formalize stimulus generation.

Chapter 15 — Talking to Multiple Objects (TLM)

- Introduces **uvm_analysis_port #(T)** — a 1-to-many broadcast port.
- Receivers use **uvm_analysis_imp** (with a *write()* method) or **uvm_subscriber #(T)** (with analysis_export).
- Monitor broadcasts: scoreboard + coverage + checker subscribe. Decouples the monitor from its consumers.
- Replaces Spear-style mailboxes.

6. Factory Pattern in 60 Seconds

Two macros register a class with the factory:

```
`uvm_object_utils(my_class) // for uvm_object descendants (transactions, sequences,
configs)
`uvm_component_utils(my_class) // for uvm_component descendants (driver, env, test, ...)
```

Two ways to create an instance:

```
// WRONG (skips factory)
my_class h = new("name");

// RIGHT (goes through factory — supports overrides)
my_class h = my_class::type_id::create("name", parent); // parent only for components
```

Two ways to override:

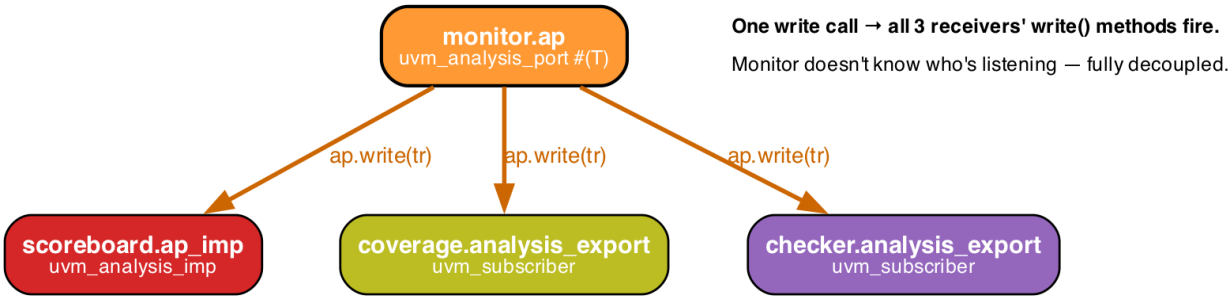
```
// Type-safe (preferred)
set_type_override_by_type(slow_driver::get_type(), fast_driver::get_type());

// String-based (good for plusargs / external config)
factory.set_type_override_by_name("slow_driver", "fast_driver");
```

Important: install overrides BEFORE the env builds. In a uvm_test::build_phase: (1) call *set_type_override*, then (2) *env = my_env::type_id::create(...)*.

7. TLM Analysis Ports in 60 Seconds

The pattern: one source port broadcasts to many subscribers.



Source side (in your monitor):

```
uvm_analysis_port #(my_transaction) ap;
function new(...);
super.new(name, parent);
ap = new("ap", this);
endfunction
task run_phase(uvm_phase phase);
forever begin
// sample bus...
ap.write(tr); // ← broadcast
end
endtask
```

Receiver side (in your scoreboard):

```
uvm_analysis_imp #(my_transaction, my_scoreboard) ap_imp;
function new(...);
super.new(name, parent);
ap_imp = new("ap_imp", this);
endfunction
function void write(my_transaction tr); // ← UVM calls this on every broadcast
// predict, compare, log
endfunction
```

Wiring (in your env's connect_phase):

```
agt.mon.ap.connect(scb.ap_imp);
agt.mon.ap.connect(cov.analysis_export); // uvm_subscriber-based
```

8. Minimum Viable Testbench — File Structure

Every file you'll write for an HW2-style skeleton:

File	Role
my_pkg.sv	Package — *import uvm_pkg::*;* + `includes for all class files
my_if.sv	Interface — bundles DUT signals + clocking block + modports
my_transaction.svh	*uvm_sequence_item* subclass — one transaction's worth of data
my_sequence.svh	*uvm_sequence* subclass — generates a stream of transactions
my_sequencer.svh	*typedef uvm_sequencer #(my_transaction) my_sequencer;*
my_driver.svh	*uvm_driver* — pulls transactions from sequencer, drives DUT
my_monitor.svh	*uvm_monitor* — passively samples bus, broadcasts via analysis port
my_agent.svh	*uvm_agent* — wraps driver + sequencer + monitor
my_scoreboard.svh	*uvm_scoreboard* — subscribes via uvm_analysis_imp, predicts + checks
my_env.svh	*uvm_env* — builds agent + scoreboard, wires their TLM connections
my_base_test.svh	*uvm_test* — builds env; common config for all tests
my_specific_test.svh	extends *my_base_test* — picks a sequence to start
tb_top.sv	Top module — clock + DUT + interface; calls *run_test()*

The package's `*include* ORDER matters:
leaves first → roots last. Specifically: transaction → sequence → sequencer → driver → monitor → agent → scoreboard → env → base_test → specific_test.

9. Common Confusions When Coming From Salemi

You saw in Salemi…	But in real UVM…
scoreboard accessing bfm directly: *@(posedge bfm.done)*	scoreboard subscribes to monitor's analysis_port. It NEVER touches the bus directly.
BFM tasks like *send_op* in the interface	Bus-driving logic lives in the **driver**. The interface is a passive signal bundle + clocking + assertions.
tester generates AND drives stimulus	Split into **sequence** (what) + **sequencer** (arbitrate) + **driver** (drive).
random_tester swapped via factory override	Different stimulus = different **sequence**. Different protocol = different **driver**.
fork ... join_none in the testbench class	UVM phases handle this. raise_objection / drop_objection in run_phase.
No agent at all in the env	A real env has at least one agent (and usually several — one per interface).
No sequence object	A test ALWAYS starts a sequence (or several). The sequence is the unit of stimulus.

10. Where to Find Progressive Exercises

You asked for a place with **exercises that build up step-by-step from simple to complex**. Here are the resources I'd actually use, ranked by suitability for your situation:

#	Resource	Format	Best for
1	**ChipVerify UVM Tutorial** chipverify.com/uvm/uvm-tutorial	Web pages, free, no signup	Reading order is the EXERCISE. Each page builds on the previous. Code snippets you can copy into EDA Playground and modify.
2	**Verification Guide UVM Tutorial** verificationguide.com/uvm/uvm-tutorial	Web pages, free	Alternative explanations to ChipVerify. Sometimes a clearer example for a topic you didn't get there.
3	**EDA Playground** edaplayground.com	Browser-based simulator, free login	Click 'Examples' → filter UVM. Hundreds of small testbenches you can fork, run, and modify in your browser. Search 'UVM minimal testbench', 'UVM hello world', etc.
4	**Verification Academy UVM Cookbook** verificationacademy.com/cookbook	Free signup, articles + videos	More reference than tutorial, but every topic has runnable code. Use it to look up specific concepts after exercises 1-3.
5	**Salemi's UVM Primer GitHub** github.com/raysalemi/uvmprimer	Source code from the book, free	Every example from your book in runnable form. Fork it, modify the testers, see what breaks.
6	**Doulos Easier UVM Code Generator** doulos.com/knowhow/systemverilog/uvm/easier-uvm	Web tool + tutorials	Generates a complete UVM skeleton from a config file. Great for studying generated code and naming conventions.

11. The Tiered Exercise Path (start here, build up)

If you want a structured roadmap rather than a website, here's a 5-tier sequence. Each tier is 30–90 minutes. **Don't skip a tier — the goal is to feel each concept in your fingers, not just your eyes.** All exercises target a tiny made-up DUT (e.g. a 4-bit counter or an adder). Use ChipVerify (resource #1 above) as your reference.

Tier 1 — Empty UVM, no DUT (~30 min)

- **Ex 1.1:** write a uvm_test that prints "Hello UVM" in run_phase. tb_top calls run_test().
- **Ex 1.2:** add a uvm_env. The test creates the env in build_phase. Print from env's build_phase.
- **Ex 1.3:** add a uvm_agent inside the env. Print from agent's build_phase. Verify the topology with uvm_top.print_topology().

Goal: see the build_phase order — top-down — and the topology print.

Tier 2 — Stimulus path (~60 min)

- **Ex 2.1:** add a uvm_sequence_item with one rand field (e.g. *rand bit [3:0] data*).
- **Ex 2.2:** add a uvm_sequence that creates 5 transactions and finishes them.
- **Ex 2.3:** add a uvm_sequencer (typedef) inside the agent. The driver prints each transaction it gets.
- **Ex 2.4:** the test starts the sequence on the agent's sequencer in run_phase.

Goal: feel the sequence → sequencer → driver TLM flow without a DUT yet.

Tier 3 — Connect to a real DUT (~60 min)

- **Ex 3.1:** write a tiny DUT (4-bit counter or 8-bit adder).
- **Ex 3.2:** write the SV interface and a clocking block. Drive a clock from tb_top.
- **Ex 3.3:** set the virtual interface in uvm_config_db from tb_top; get it in the driver's build_phase.
- **Ex 3.4:** driver pokes the DUT through the clocking block on each transaction.

Goal: prove you can drive real hardware through the UVM stack.

Tier 4 — Observability (~60 min)

- **Ex 4.1:** add a uvm_monitor that samples the bus and prints what it sees.
- **Ex 4.2:** add a uvm_analysis_port to the monitor; have it ap.write(tr) on every observation.
- **Ex 4.3:** add a uvm_scoreboard with uvm_analysis_imp; print every transaction it receives.
- **Ex 4.4:** implement a reference model in the scoreboard. Compare predicted vs observed; print PASS/FAIL.

Goal: feel the analysis path and the 1-to-many broadcast pattern.

Tier 5 — UVM-specific muscles (~90 min)

- **Ex 5.1:** add a uvm_subscriber for coverage with a small covergroup.
- **Ex 5.2:** add a SECOND test class with different stimulus (e.g. directed corner cases). Pick at runtime via +UVM_TESTNAME.

- **Ex 5.3:** create two driver classes (slow + fast) and use set_type_override to swap them at the test level.
- **Ex 5.4:** implement do_copy / do_compare / convert2string on your sequence_item. Test by copying, comparing, and printing two transactions.

Goal: graduate to multi-test, override-driven UVM — the same patterns Salemi ch.13 covers.

Tier 6 — Re-read Salemi 9–15 with the new lens

- After Tier 5 you'll recognize WHY each Salemi pattern exists.
- Second-pass reading is when the chapters click. First-pass confusion is normal.
- Then: tackle your week-4 homework HW2-HW4 with confidence.

Last thing: being lost after one read of UVM is the default. You're not behind — you're in the part where it feels chaotic before it clicks. **Build tiny, run, repeat.** The second read is when it clicks.

— end of consolidation —